Department of Computer and Software Engineering- ITU

CE101L: Object Oriented Programming Lab

Course Instructor: Nadir Abbas	**Dated: 25/03/2025**
Teaching Assistant: Zainab, Nahal, Bilal	**Semester: Spring 2025**
Session: 2024-2028	**Batch: BSCE24**

Lab 8. Matchmaking System Using Object-Oriented Programming Concepts

Name	Roll number	Obtained Marks/35

Checked on: _____

Signature: _____

Objective

The objective of this lab is to help students understand and apply concepts of oop.

Equipment and Component

Component Description	Value	Quantity
Computer	Available in lab	1

Conduct of Lab

1. Students are required to perform this experiment individually.
2. In case the lab experiment is not understood, the students are advised to seek help from the course instructor, lab engineers, assigned teaching assistants (TA) and lab attendants.

Theory and Background

In Object-Oriented Programming (OOP), **relationships between classes** define how objects interact with each other. The key relationships covered in this lab are **Association, Aggregation, and Composition**.

1. Aggregation

Aggregation is a **"has-a"** relationship where one class contains an object of another class, but the contained object **can exist independently**.

- **Example:** A **Car** has an **Engine**, but the **Engine** can exist separately and be used in other cars.

2. Composition

Composition is a **strong form of Aggregation**, where one class **owns** objects of another class, and those objects **cannot exist independently**.

- **Example:** A **Building** consists of multiple **Rooms**, but a **Room** cannot exist without a **Building**.

These concepts help in designing modular, reusable, and well-structured object-oriented systems.

Tasks

Task 1: Accept the assignment posted in Google Classroom and after accepting clone the repository to your computer for this ensure you have logged into github app with your account.

Task 2: Solve the given problems written after task instructions, write code through IDE like CLion

Task 3: Ensure your code/solution is in the cloned folder.

Task 4: Commit and Push the changes through the Github App

Write code in functions, in files **functions.cpp**, **functions.h** and **main.cpp** after completing each part, verify through running code using **"make run"** on Cygwin.

1. Class Person (Encapsulation & Operator Overloading)

Data Members (Private)

- name (string): Person's name
- age (int): Person's age
- preference (string): Preferred partner's characteristic
- bondScore (int): Relationship strength
- isMarried (bool): True if married, False otherwise

Member Functions

- **Constructors**
  - Person(): Default constructor
  - Person(string name, int age, string preference, int bondScore, bool isMarried): Constructor with full details
  - Person(string name, int age): Constructor for partial details
- **Getters**
  - string getName() const: Returns the name
  - int getAge() const: Returns the age
  - string getPreference() const: Returns the preference
  - int getBondScore() const: Returns the bond score
  - bool getMaritalStatus() const: Returns marital status
- **Setters**
  - void setName(string name): Sets the name
  - void setAge(int age): Sets the age
  - void setPreference(string preference): Sets the preference
  - void setBondScore(int bondScore): Sets the bond score
  - void setMaritalStatus(bool status): Sets the marital status
- **Operator Overloading**
  - bool operator+(const Person& other): Matches two people if preferences match
  - void operator++(): Increases bond score when two people interact

2. Class BasicPerson & PremiumPerson (Inheritance & Polymorphism)

Inheritance:

- BasicPerson inherits from Person
- PremiumPerson inherits from Person

Constructors

- BasicPerson(): Default constructor
- BasicPerson(string name, int age, string preference, int bondScore, bool isMarried): Constructor with details
- PremiumPerson(): Default constructor
- PremiumPerson(string name, int age, string preference, int bondScore, bool isMarried): Constructor with details

Overridden Function

- void findMatch(): Prints "Basic Person match finding" for BasicPerson and "Premium Person match finding" for PremiumPerson

3. Class Nikah (Composition - Part of Marriage)

Data Members (Private)

- mahr (int): Dowry amount
- witnesses (string[2]): Two witnesses of the marriage
- nikahStatus (bool): True if Nikah is completed

Member Functions

- **Constructor**
  - Nikah(int mahr, string witness1, string witness2): Initializes Nikah with details
- **Getters**
  - int getMahr() const: Returns mahr amount
  - string getWitness(int index) const: Returns witness at given index
  - bool isNikahFinalized() const: Returns Nikah status
- **Setters**
  - void setMahr(int mahr): Sets mahr amount
  - void setWitness(int index, string witness): Sets witness at given index
  - void setNikahStatus(bool status): Sets Nikah status
- **Functions**
  - void finalizeNikah(): Completes Nikah by setting nikahStatus to true
  - void cancelNikah(): Cancels the Nikah

4. Class Marriage (Composition - Uses Nikah)

Data Members (Private)

- groom (Person*): Pointer to groom
- bride (Person*): Pointer to bride
- nikah (Nikah): Nikah contract associated with marriage

Member Functions

- **Constructor**
  - Marriage(Person* groom, Person* bride, Nikah nikah): Initializes marriage with details
- **Getters**
  - Person* getGroom() const: Returns groom
  - Person* getBride() const: Returns bride
  - Nikah getNikah() const: Returns Nikah contract

- **Setters**
    - void setGroom(Person* groom): Sets groom
    - void setBride(Person* bride): Sets bride
    - void setNikah(Nikah nikah): Sets Nikah contract
- **Functions**
    - void completeMarriage(): Finalizes marriage by completing Nikah.

5. Class MarriageBureau (Aggregation - Manages Marriages)

Data Members (Private)

- marriages (Marriage*[100]): Array of marriages
- marriageCount (int): Number of marriages

Member Functions

- **Constructor**
    - MarriageBureau(): Initializes marriage bureau
- **Functions**
    - void registerMarriage(Marriage* newMarriage): Adds a marriage to the bureau
    - int getMarriageCount() const: Returns the number of marriages

6. Class MatchMakingSystem (Manages Everything & Handles File I/O)

Data Members (Private)

- basicPeople (BasicPerson[100]): Array to store BasicPerson objects
- premiumPeople (PremiumPerson[100]): Array to store PremiumPerson objects
- basicCount (int): Number of BasicPerson objects
- premiumCount (int): Number of PremiumPerson objects
- bureau (MarriageBureau): Marriage bureau to manage marriages

Member Functions

- **Constructor**
    - MatchMakingSystem(): Initializes matchmaking system
- **Functions**
    - void addBasicPerson(BasicPerson p): Adds a basic person to the system
    - void addPremiumPerson(PremiumPerson p): Adds a premium person to the system
    - void removePerson(string name): Removes a person by name first from basic then premium
    - Person* matchPeople(int& matchCount): Returns an array of matched persons
    - void increaseBond(string name1, string name2): Increases bond score between two people
    - void saveToFile(): Saves people to a JSON file
    - void loadFromFile(): Reads people from a JSON file
- **Getter Functions**:
    - BasicPerson* getBasicPeople(): Returns the array of basic people.
    - PremiumPerson* getPremiumPeople(): Returns the array of premium people.
    - int getBasicCount(): Returns the number of basic people.
    - int getPremiumCount(): Returns the number of premium people.

Assessment Rubric for Lab

Performance metric	CLO	Able to complete the task over 80% (4-5)	Able to complete the task 50-80% (2-3)	Able to complete the task below 50% (0-1)	Marks
1. Realization of experiment	1	Executes without errors excellent user prompts, good use of symbols, spacing in output. Through testing has been completed.	Executes without errors, user prompts are understandable, minimum use of symbols or spacing in output. Some testing has been completed.	Does not execute due to syntax errors, runtime errors, user prompts are misleading or non-existent. No testing has been completed.	
2. Conducting experiment	1	Able to make changes and answered all questions.	Partially able to make changes and few incorrect answers.	Unable to make changes and answer all questions.	
3. Computer use	2	Document submission timely.	Document submission late.	Document submission not done.	
4. Teamwork	3	Actively engages and cooperates with other group member(s) in effective manner.	Cooperates with other group member(s) in a reasonable manner but conduct can be improved.	Distracts or discourages other group members from conducting the experiment	
5. Laboratory safety and disciplinary rules	3	Code comments are added and does help the reader to understand the code.	Code comments are added and does not help the reader to understand the code.	Code comments are not added.	
6. Data collection	3	Excellent use of white space, creatively organized work, excellent use of variables and constants, correct identifiers for constants, No line-wrap.	Includes name, and assignment, white space makes the program fairly easy to read. Title, organized work, good use of variables.	Poor use of white space (indentation, blank lines) making code hard to read, disorganized and messy.	
7. Data analysis	4	Solution is efficient, easy to understand, and maintain.	A logical solution that is easy to follow but it is not the most efficient.	A difficult and inefficient solution.	
Total (out of 35):					